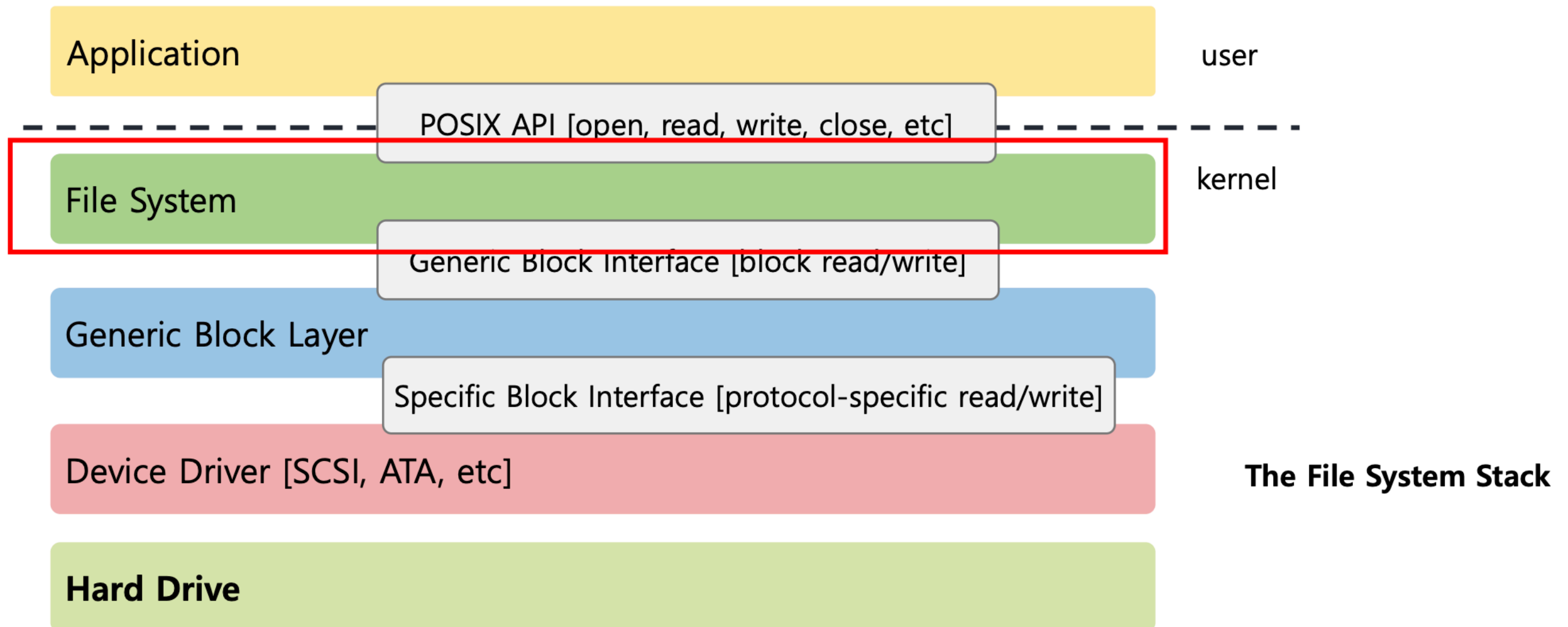


File Systems

Thierry Sans

File System Abstraction

File system specifics of which disk class it is using
It issues block read/write requests to the **generic block layer**



Provide an abstraction

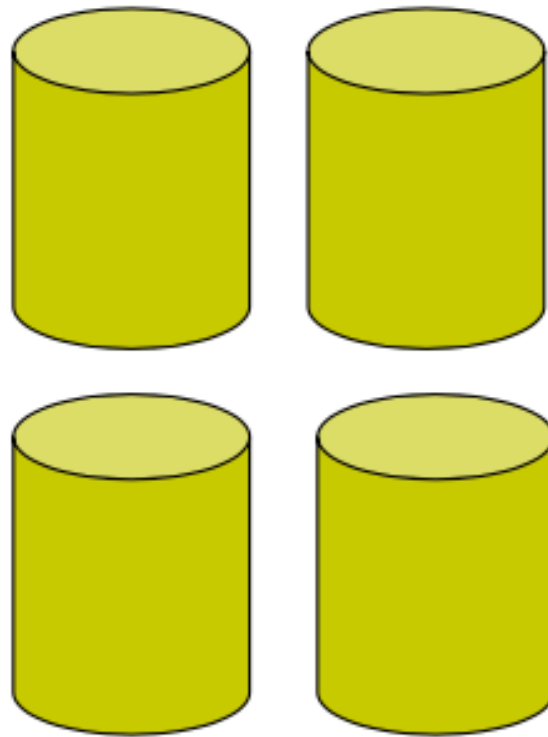
Reality

Hard Disks

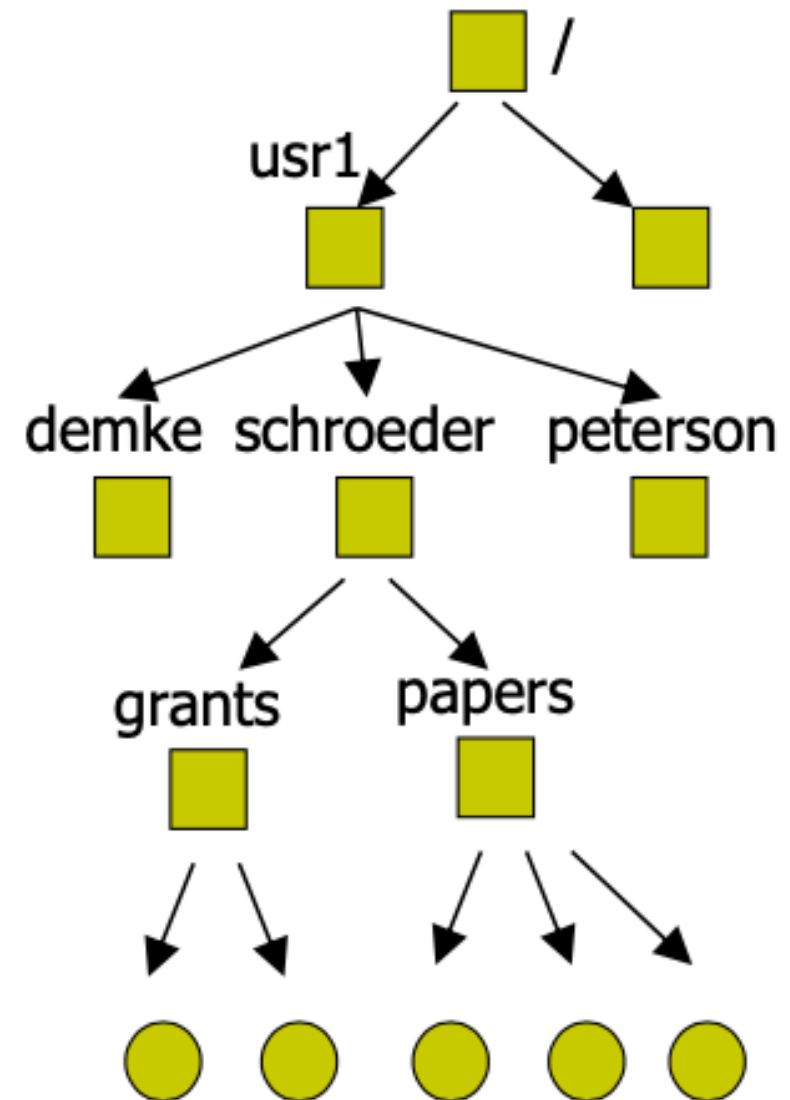
Read block #42



Write block #14



Abstraction



Goals

- Implement an abstraction (files) for secondary storage
- Organize files logically (directories)
- Permit sharing of data between processes, people, and machines
- Protect data from unwanted access (security)

Files

File - named bytes on disk that encapsulate data with some properties: contents, size, owner, last read/write time, protection, etc.

A file can also have a type

- Understood by the file system: block device, character device, link, FIFO, socket, etc.
- Understood by other parts of the OS or runtime libraries: text, image, source, compiled libraries (Unix `.so` and Windows `.dll`), executable, etc.

A file's type can be encoded in its name or contents

- Windows encodes type in name: `.com`, `.exe`, `.bat`, `.dll`, `.jpg`, etc.
- Unix encodes type in contents: magic numbers, initial characters (e.g., `#!` for shell scripts)

File Access Method

Sequential access (used by file systems - most common)

read bytes one at a time, in order (read/write next)

Random access (used by file systems)

random access given block/byte number (read/write bytes at offset n)

Indexed access (used by databases)

- file system contains an index to a particular field of each record in a file
- reads specify a value for that field and the system finds the record via the index

Record access (used by databases)

- file is array of fixed-or-variable-length records
- read/written sequentially or randomly by record number

Basic File operations

Unix

- `create(name)`
- `open(name, how)`
- `read(fd, buf, len)`
- `write(fd, buf, len)`
- `sync(fd)`
- `seek(fd, pos)`
- `close(fd)`
- `unlink(name)`

Windows

- `CreateFile(name, CREATE)`
- `CreateFile(name, OPEN)`
- `ReadFile(handle, ...)`
- `WriteFile(handle, ...)`
- `FlushFileBuffers(handle, ...)`
- `SetFilePointer(handle, ...)`
- `CloseHandle(handle, ...)`
- `DeleteFile(name)`
- `CopyFile(name)`
- `MoveFile(name)`

How to Track File's Data

Disk management

- Need to keep track of where file contents are on disk
- Must be able to use this to map byte offset to disk block
- Structure tracking a file's blocks is called an index node or **inode**
- Inodes must be stored on disk, too

Things to keep in mind while designing file structure

- Most files are small
- Much of the disk is allocated to large files
- Many of the I/O operations are made to large files
- Want good sequential and good random access (what do these require?)

Different File Systems

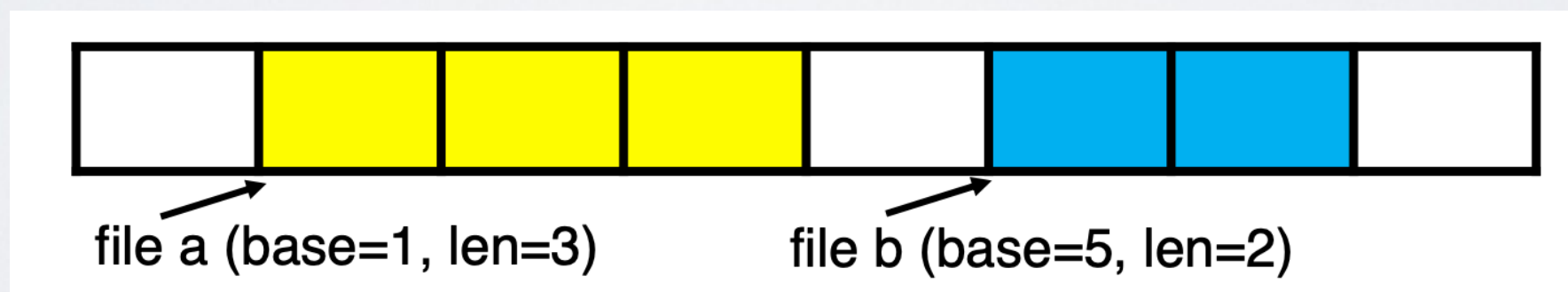
- **Straw Man** (Pintos otherwise not used in practice)
- **FAT** File Allocation Table (DOS and old Windows)
- **FFS** Unix Fast File System (all modern operating systems)

Straw Man

Straw Man #1: Contiguous Allocation

"Extent-based" - allocate files like segmented memory

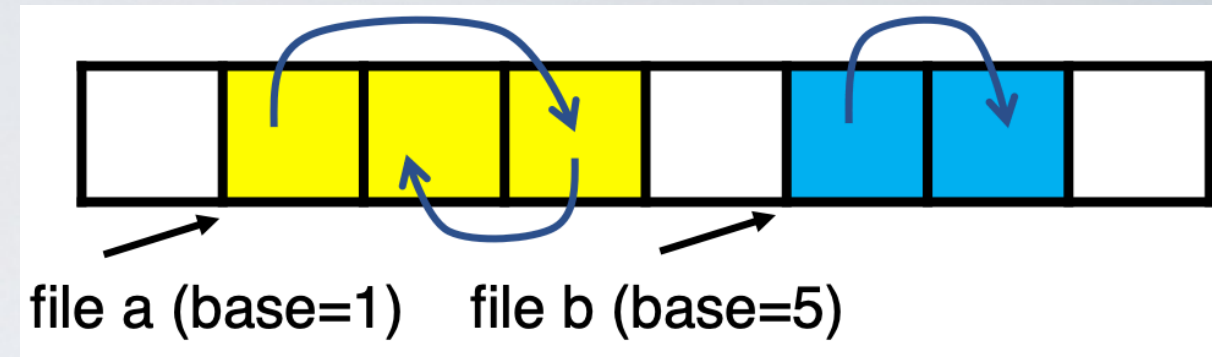
- When creating a file, make the user pre-specify its length and allocate all space at once
- Inode contents : location and size



✓ Simple, fast access, both sequential and random

● External fragmentation (similar to VM)

Straw Man #2 : Linked Files



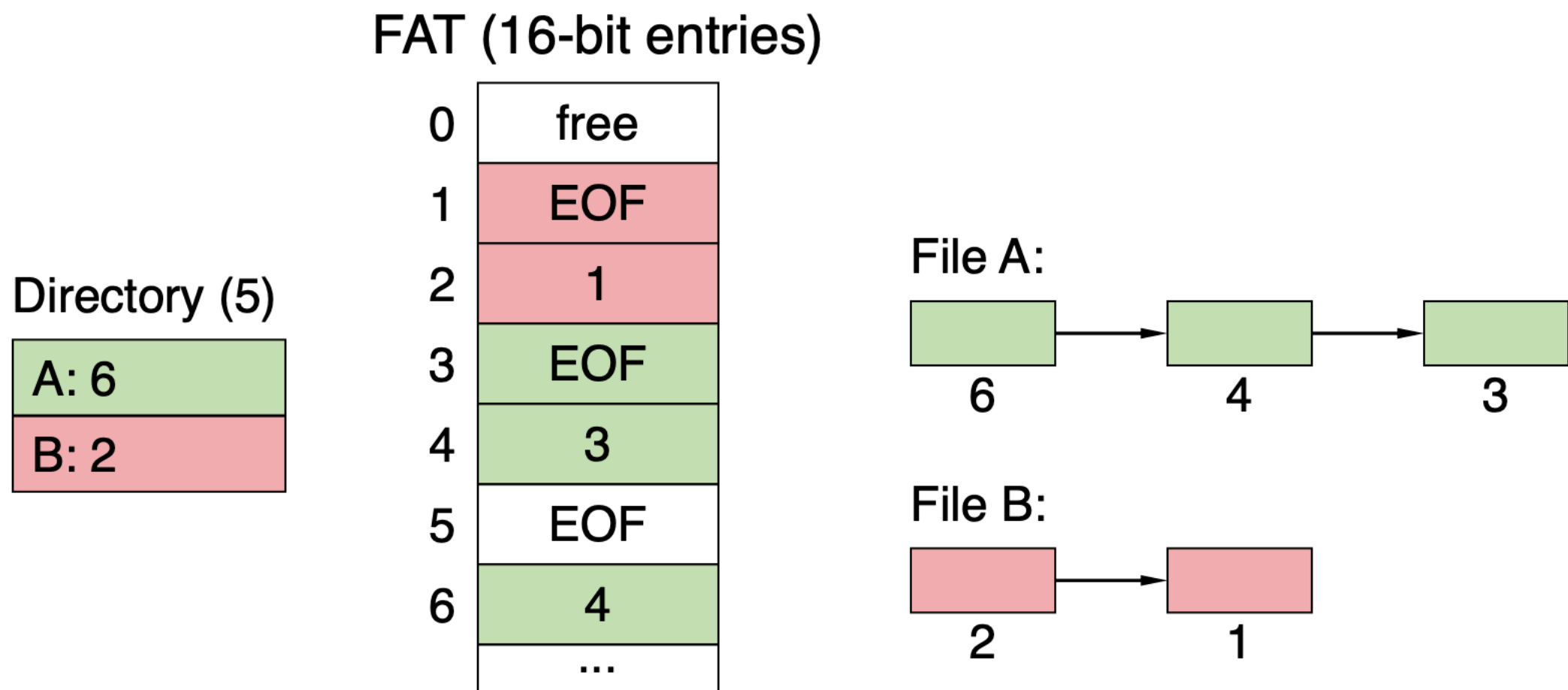
Basically a linked list on disk

- Keep a linked list of all free blocks
 - Inode contents : a pointer to file's first block
 - In each block, keep a pointer to the next one
- ✓ Easy dynamic growth & sequential access, no fragmentation
- Linked lists on disk a bad idea because of access times
Random very slow (e.g., traverse whole file to find last block)
Pointers take up room in block, skewing alignment

FAT

DOS FAT (simplified)

- ➔ Linked files with key optimization: puts links in fixed-size "file allocation table" (FAT) rather than in the blocks
- Still do pointer chasing



About FAT

Given entry size = 16 bits (initial FAT16 in MS-DOS 3.0), what's the maximum size of the FAT? **65,536**

Given a 512 byte block, what's the maximum size of FS? **32MB**

What is the space overhead ?

2 bytes / 512 byte block = ~ 0.4%

How to protect against errors?

Create duplicate copies of FAT on disk

(state duplication a very common theme in reliability)

Where is root directory?

Fixed location on disk

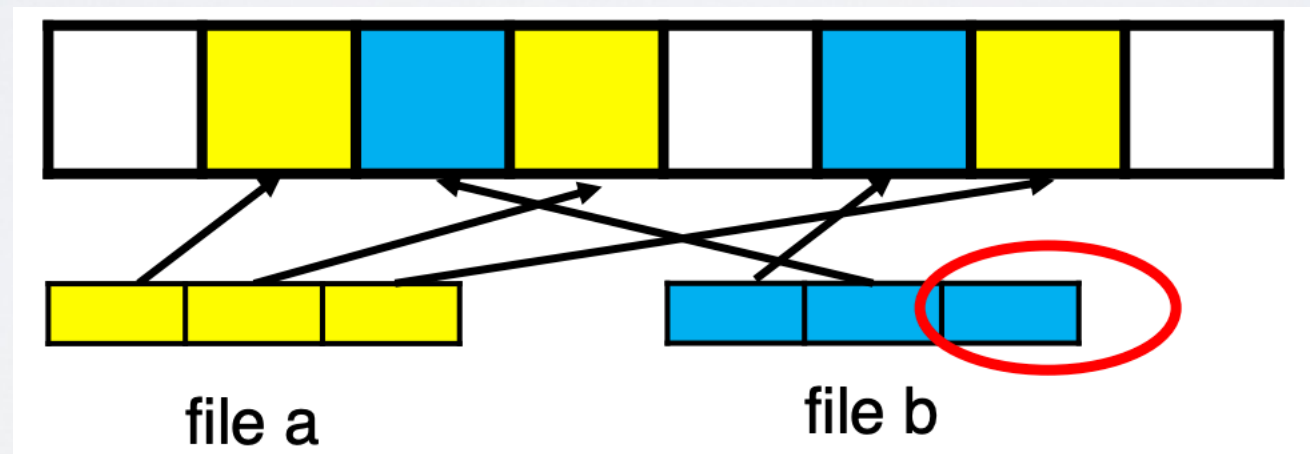
FAT	FAT (opt)	Root dir	...
-----	-----------	----------	-----

Unix Fast File System

Another Approach : Indexed Files

Each file has a table holding all of its block pointers

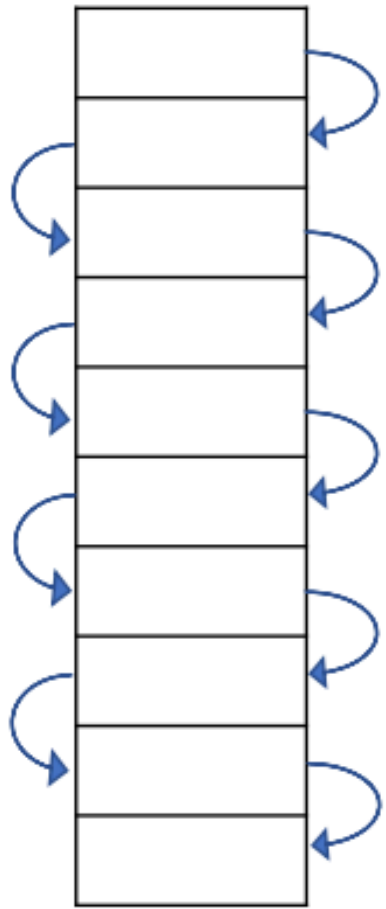
- Max file size fixed by table's size
- Allocate table to hold file's block pointers on file creation
- Allocate actual blocks on demand using free list



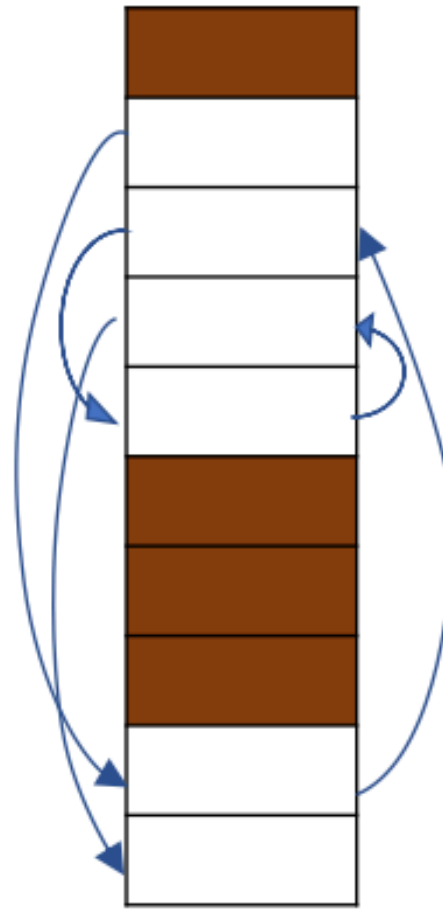
✓ Both sequential and random access easy

⦿ Mapping table requires large chunk of contiguous space

Problem with Unorganized Freelist



Initial performance good



Get worse over time

Measurement:

- New FS: **17.5%** of disk bandwidth
- Few weeks old: **3%** of disk bandwidth

● Leads to random allocation of sequential file blocks overtime

Solution - Bitmaps

Periodical compact/defragment disk

- locks up disk bandwidth during operation

Keep adjacent free blocks together on freelist

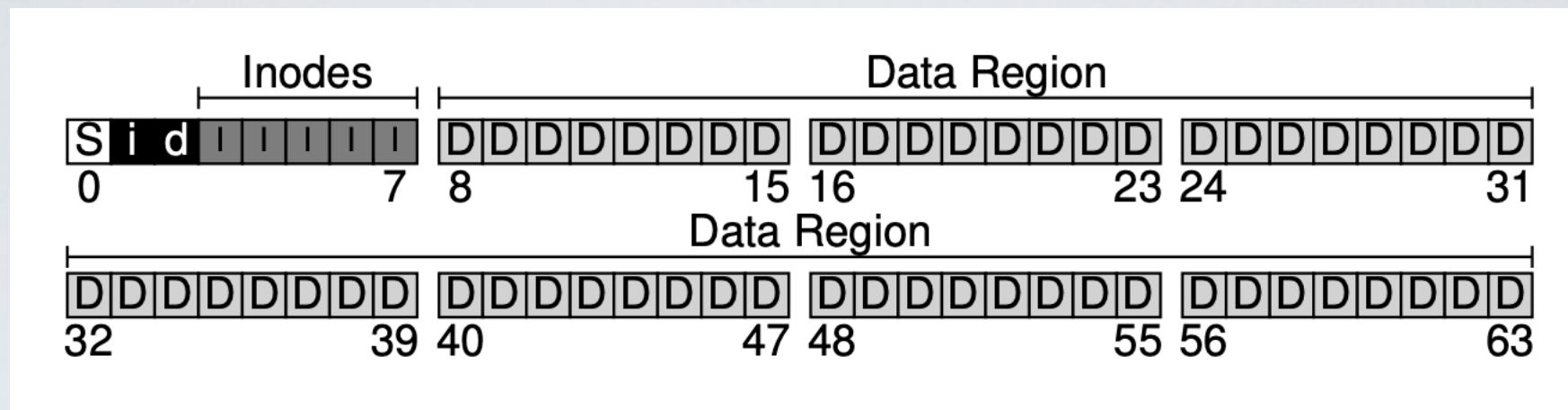
- costly to maintain

➔ FFS - bitmap of free blocks (same idea as Indexed File System)

- Each bit indicates whether block is free
e.g., 10101011111100000111111000101100
- Easier to find contiguous blocks
- Small, so usually keep entire thing in memory
- Time to find free blocks increases if fewer free blocks

Bits in map	XXXX	XXOO	OOXX	OOOO
Fragment numbers	0-3	4-7	8-11	12-15
Block numbers	0	1	2	3

Unix File System



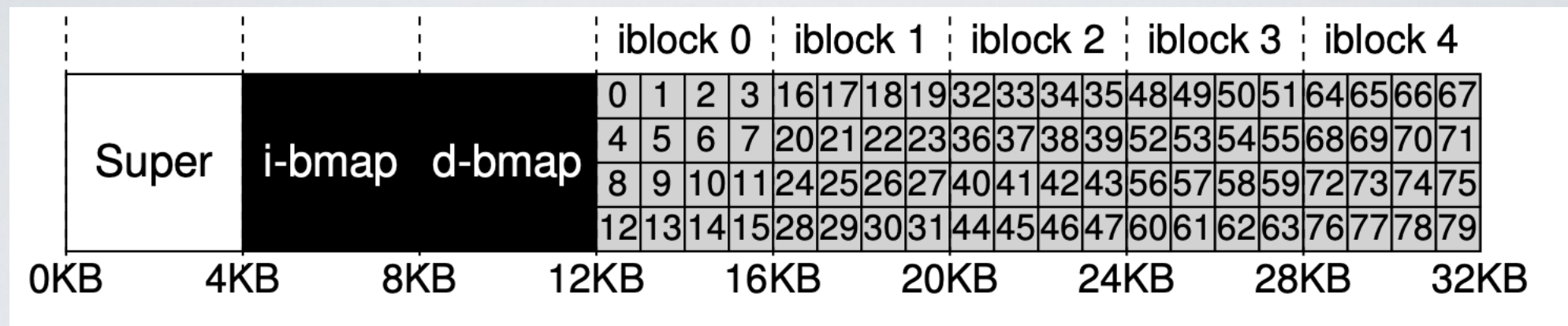
The disk is (physically) divided into sectors (usually 512 bytes per sector)

The file system is (logically) divided into blocks (e.g., 4 KB per block)

➔ Disk space is allocated in granularity of blocks

1. The data blocks "D" stored files (and directories) content
2. The inodes blocks "I" stores the inode table
3. The data bitmap "d" block d tracks which data block is free or allocated (one bit per block on the disk)
4. The inode bitmap "i" block i tracks which inode is free or allocated (one bit per inode)
5. The Superblock "S" (a.k.a Master Block or partition control block) contains:
 - a magic number to identify the file system type
 - the number of blocks dedicated to the two bitmaps and inodes

The Inode Table



- **Physical Disk capacity in our example** (64 blocks of 4KB each)
 $4 \times 64 = 256 \text{ KB}$
- **Logical capacity (8 blocks are reserved)**
 $4 \times 56 = 224 \text{ KB}$ (the actual data storage space)
- **Maximum number of inodes** (each inode is 256 bytes)
 $(5 * 4 * 1024) / 256 = 80 \text{ inodes}$ (i.e max number of files)
- **Size of the inode bitmap** (1 bit per inode)
 $1 \times 80 \text{ inodes} = 80 \text{ bits}$ (out of 32K bits)
- **Size of the data bitmap** (1 bit per storage block)
 $1 \text{ bit} \times 56 \text{ blocks} = 56 \text{ bits}$ (out of 32K bits, max data storage 128 MB)

Unix Inode (simplified)

Size	Name	Description
2	mode	can the file be read/written/executed
2	uid	file owner id
4	size	the file size in bytes
4	time	time the file was last accessed
4	ctime	time when the file created
4	mtime	time when the file was last modified
4	dtime	time when the inode was deleted
2	gid	file group owner id
2	links_count	number of hard links pointing to this file
4	blocks	the number of blocks allocated to this file
60	block	disk pointers (15 in total)
4	file_acl	ACL permissions
4	dir_acl	ACL permissions

Block pointers and maximum file size

So far, each inode has 15 block pointers

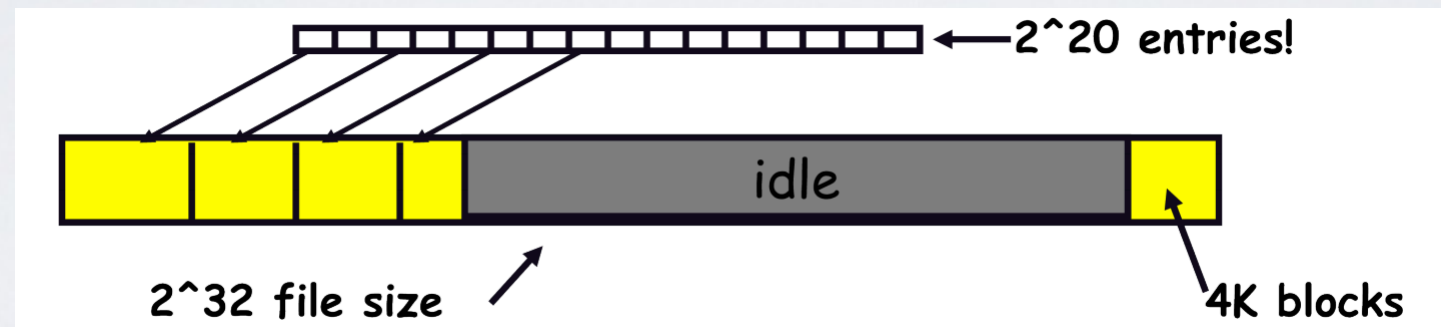
➡ The maximum file size can be $15 * 4 \text{ KB} = 60 \text{ KB}$ (only?!)

- ⦿ Should we increase the number of block pointers to increase the file size?

More issues with indexed Files

Large file size with lots of unused entries means

- The mapping table requires large chunk of contiguous space



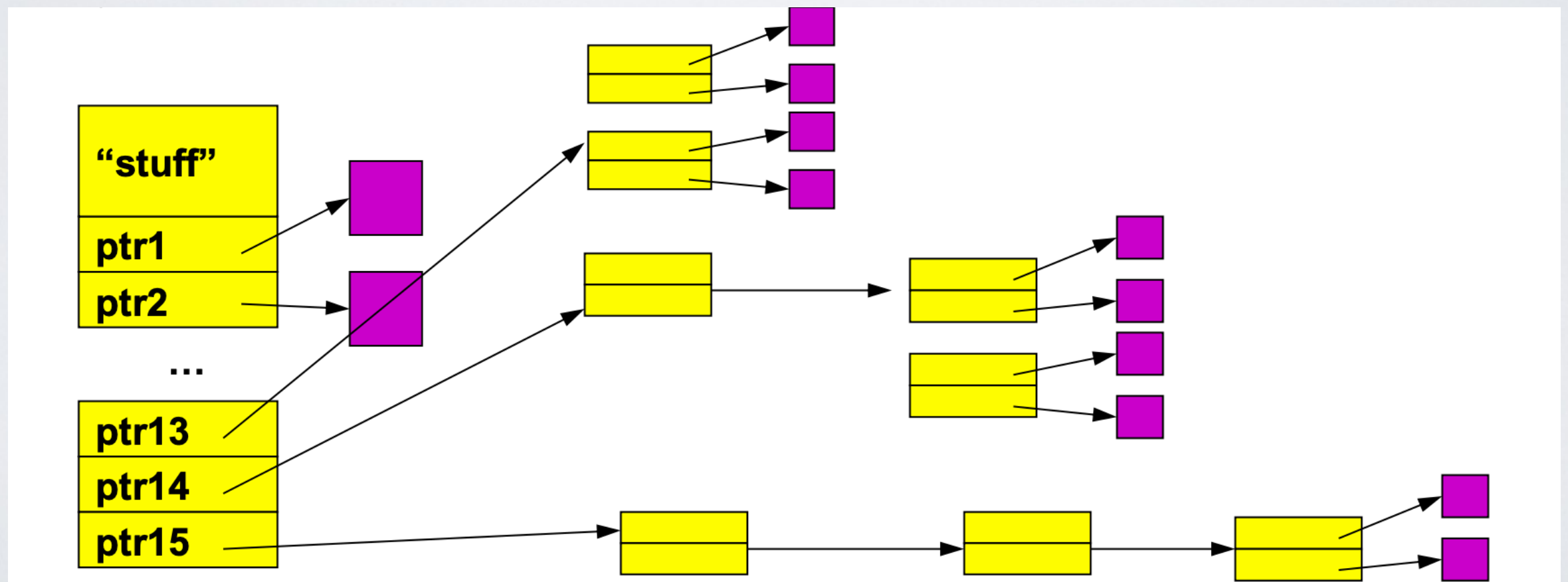
- ➔ Solution : mapping table structured as a multi-level index array



- but ... you know the story (similar to VM two-level page lookup)

Multi-level indexed files : Unix File System

- First 12 pointers are direct blocks
solve problem of first blocks access slow
- Then single, double, and triple indirect block pointers



File size with multi-level indexed files

File size using 12 direct blocks : $12 \times 4 \text{ KB} = 48 \text{ KB}$

➡ Adding single indirect block : $(12 + 1024) \times 4 \text{ KB} \sim 4 \text{ MB}$

➡ Adding a double indirect block :
 $(12 + 1024 + 1024^2) \times 4 \text{ KB} \sim 4 \text{ GB}$

➡ Adding a triple indirect block :
 $(12 + 1024 + 1024^2 + 1024^3) \times 4 \text{ KB} \sim 4 \text{ TB}$

Rationale behind multi-level index files

- **Most files are small**
~2K is the most common size
- **Average file size is growing**
Almost 200K is the average
- **Most bytes are stored in large files**
A few big files use most of space
- **File systems contains lots of files**
Almost 100K on average
- **File systems are roughly half full**
Even as disks grow, file systems remain ~50% full
- **Directories are typically small**
Many have few entries; most have 20 or fewer

Directories and Symbolic Links

Directories

Directories serve two purposes

- For users, they provide a structured way to organize files by using digestible names rather than inode numbers directly
- For the File System, they provide a convenient naming interface that allows the separation of logical file organization from physical file placement on the disk

Basic Directory Operations

Unix

➔ Directories implemented in file and a C runtime library provides a higher-level abstraction for reading directories

- `opendir(name)`
- `readdir(DIR)`
- `seekdir(DIR)`
- `closedir(DIR)`

Windows

➔ Explicit dir operations

- `CreateDirectory(name)`
- `RemoveDirectory(name)`
- `FindFirstFile(pattern)`
- `FindNextFile()`

A Short History of Directories

Approach 1 : Single directory for entire system

- Put directory at known location on disk
- Directory contains `hname`, `inumber` pairs
- If one user uses a name, no one else can
- Many ancient personal computers work this way

Approach 2 : Single directory for each user

- Still clumsy, and 1s on 10,000 files is a real pain

Approach 3 : Hierarchical name spaces

- Allow directory to map names to files or other directories
- File system forms a tree (or graph, if links allowed)
- Large name spaces tend to be hierarchical
(ip addresses, domain names, scoping in programming languages, etc.)

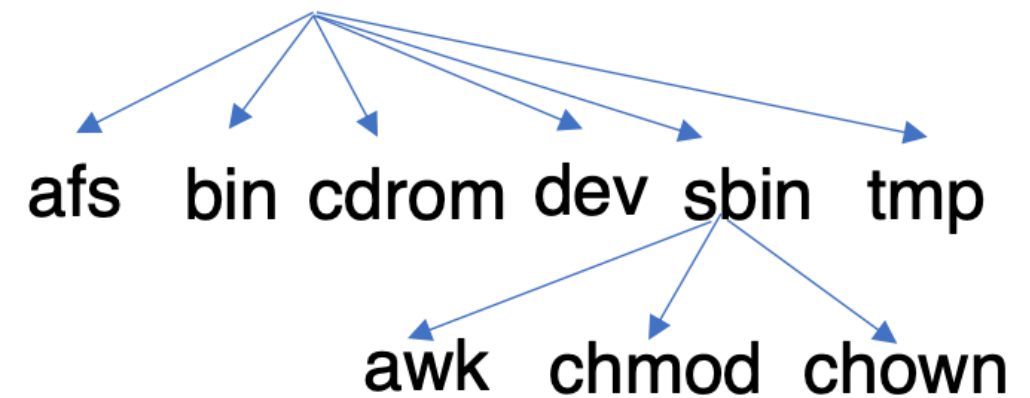
Hierarchical Directory

➔ Used since CTSS (1960s)
Unix picked up and used really nicely

Directories stored on disk just like regular files

- Special inode type byte set to directory
- User's can read just like any other file
- Only special syscalls can write
- Inodes at fixed disk location
- File pointed to by the index may be another directory
- Makes FS into hierarchical tree

✓ Simple, plus speeding up file ops speeds up dir ops!



<name,inode#>

<afs,1021>

<tmp,1020>

<bin,1022>

<cdrom,4123>

<dev,1001>

<sbin,1011>

...

Naming Magic

Bootstrapping

Root directory always inode #2 (0 and 1 historically reserved)

Special names

- Root directory : "/"
- Current directory : "."
- Parent directory : ".."

Some special names are provided by shell, not FS

- User's home directory : "~"
- Globing : "foo.*" (expands to all files starting "foo.")

Using the given names, only need two operations to navigate the entire name space

- `cd name` : move into (change context to) directory name
- `ls` : enumerate all names in current directory (context)

Unix inodes and path search

Unix inodes are not directories

- Inodes describe where on the disk the blocks for a file are placed
- Directories are files, so inodes also describe where the blocks for directories are placed on the disk

Directory entries map file names to inodes

1. To open `"/one"`, use Master Block to find inode for `"/"` on disk
2. Open `"/"`, look for entry for `"one"`
3. This entry gives the disk block number for the inode for `"one"`
4. Read the inode for `"one"` into memory
5. The inode says where first data block is on disk
6. Read that block into memory to access the data in the file

Default Context : Working Directory

Cumbersome to constantly specify full path names

- In Unix, each process has a "current working directory" (cwd)
- File names not beginning with "/" are assumed to be relative to cwd; otherwise translation happens as before

Shells track a default list of active contexts

- A "search path" for programs you run
- Given a search path A:B:C, the shell will check in A, then B, then C
- Can escape using explicit paths: "./foo"

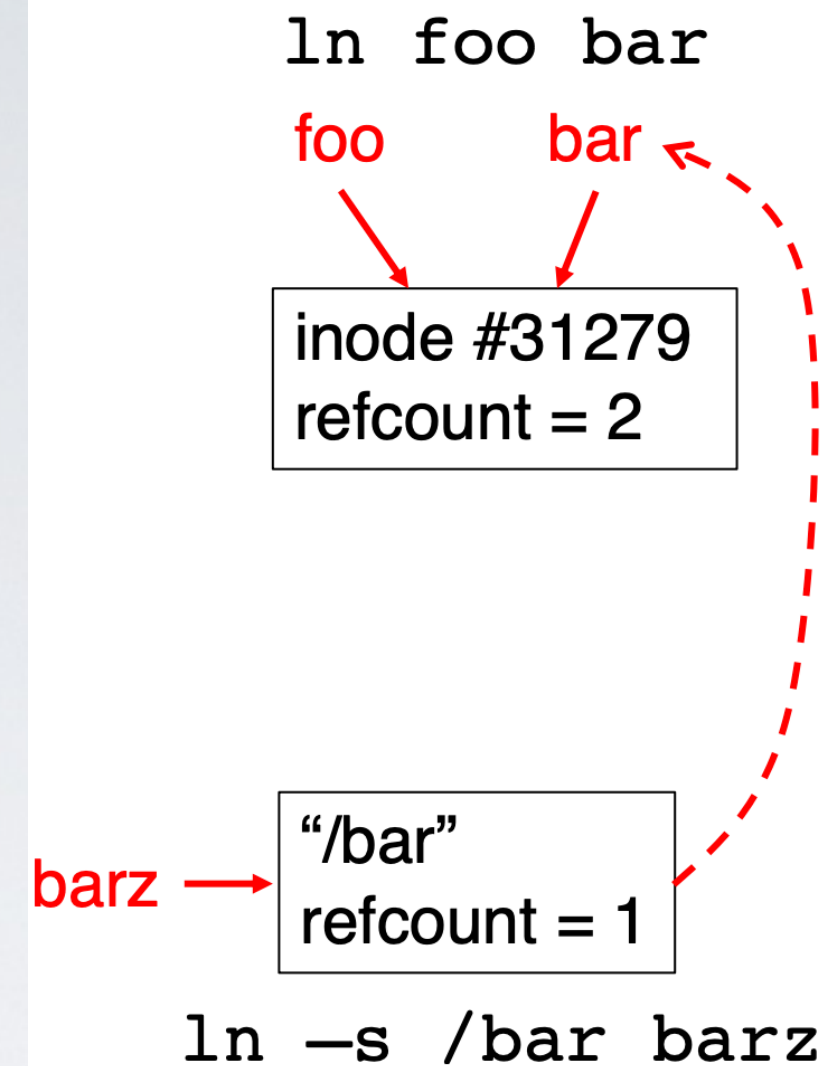
Hard and Soft Links (synonyms)

More than one dir entry can refer to a given file

- Hard link creates a synonym for file
- Unix stores count of pointers ("hard links") to inode
- If one of the links is removed (e.g., rm), the data are still accessible through any other link that remains
- If all links are removed, the space occupied by the data is freed

Soft symbolic links = synonyms for names

- Point to a file/dir name, but object can be deleted from underneath it (or never exist)
- Unix implements like directories: inode has special "symlink" bit set and contains name of link target
- When the file system encounters a soft link it automatically translates it (if possible).



Improving Reliability

with Log-Structured File system (LFS)
and Journaling File System (ext3)

What happen when power loss or system crash?

- ✓ Sectors (but not a block) are **written atomically** by the hard drive device
- ⦿ But an FS operation might modify several sectors
 - modify metada blocks (free bitmaps and inodes)
 - modify data blocks
- ➡ Crash-consistency problem
a crash has a high chance of corrupting the file system

Solution 1 - Unix fsck (File System Checker)

When system boot, check system looking for inconsistencies
e.g. inode pointers and bitmaps, directory entries and inode reference counts

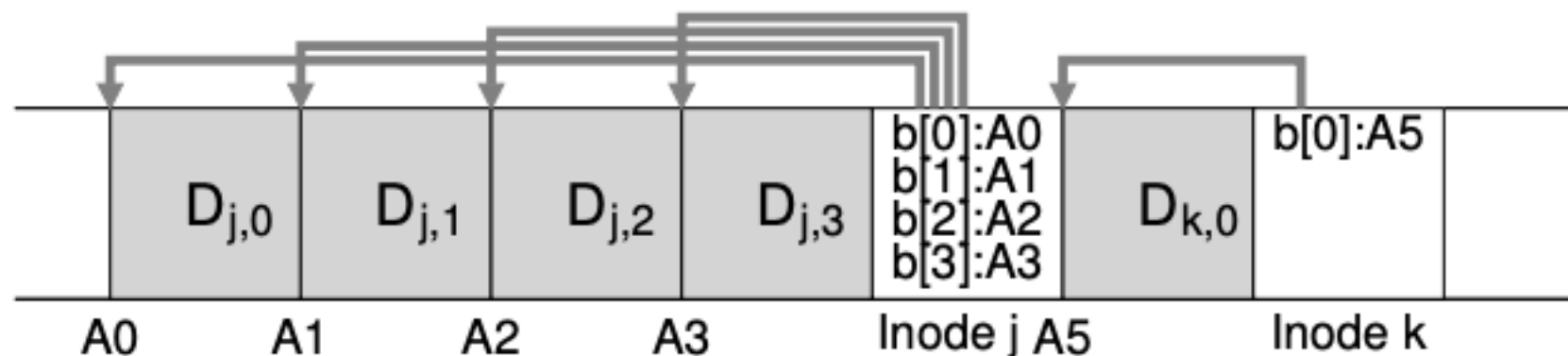
➡ Try to fix errors automatically

- ⦿ Cannot fix all crash scenarios
- ⦿ Poor performance
 - Sometimes takes hours to run on large disk volumes
 - Does fsck have to run upon every reboot?
- ⦿ Not well-defined consistency

Solution 2 - Log Structure File System (LFS) or (Copy-On-Write Logging)

Idea - treat disk like a tape-drive

1. Buffer all data (including inode) in memory segment
 2. Write buffered data to new segment on disk in a sequential log
- ➔ Existing data is not overwritten
Segment is always written in free location
- ✓ Best performance from disk for sequential access



LFS - how to find the inode table?

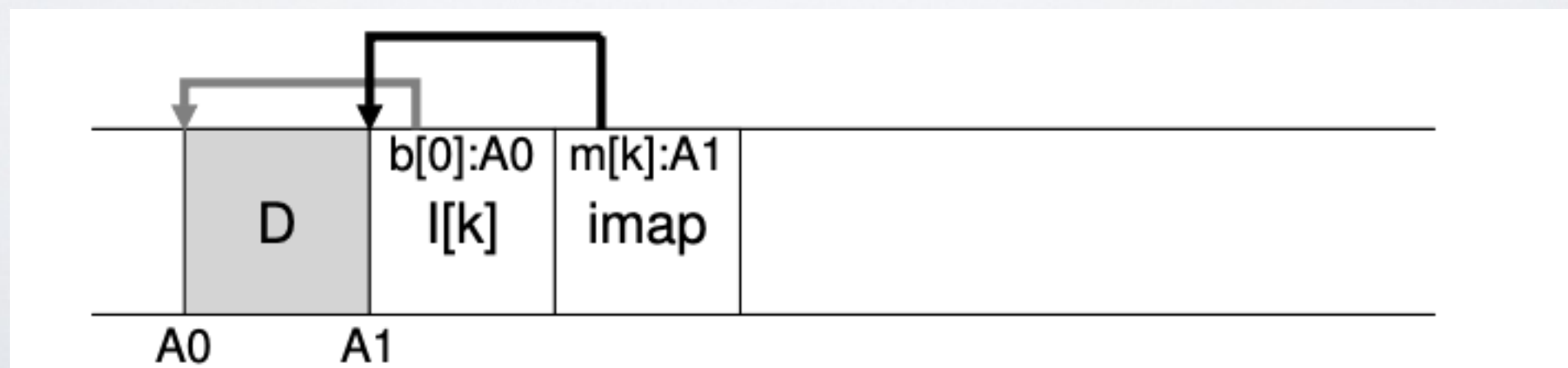
Original Unix File System

the inode table is placed at fixed location

Log-structured File System

the inode table is split and spread-out on the disk

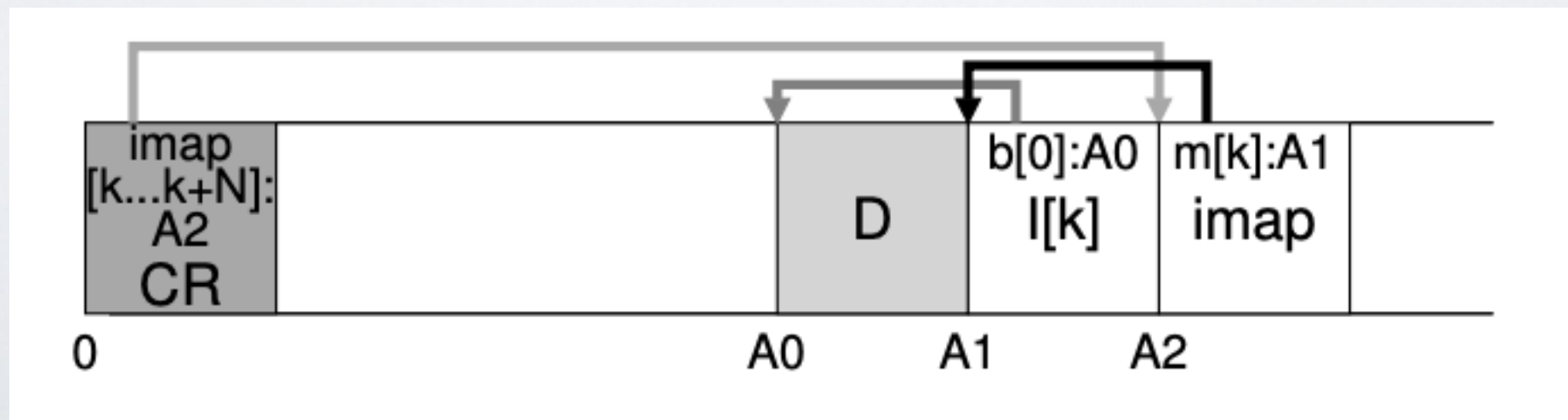
- ➔ LFS requires an inode map (imap) to map the inode number with its location on disk



LFS - how to find the inode map?

The OS must have some fixed and known location on disk to begin a file lookup

- ➔ The check-point region (CR) contains a pointer to the latest pieces of the inode map
- ✓ The CR is updated periodically (every 30 sec or so) to avoid degrading the performances



LFS - Crash recovery

The check-point region (CR) must be updated atomically

➔ LFS keeps two CRs and writing a CR is done in 3 steps

1. writes out the header with a timestamp #1

2. writes the body of the CR

3. writes one last block with another timestamp #2

✓ Crash can be detected if timestamp #1 is after #2

✓ LFS will always choose the most recent and valid CR

✓ All logs written after a successful CR update will be lost in case of a crash

LFS - Disk Cleaning (a.k.a Garbage Collection)

LFS leaves old version of file structures on disk

- ➡ LFS keeps information of the version of each segment and runs a disk cleaning process
 - A cleaning process removes old versions by compacting contiguous blocks in memory
 - That cleaning process runs when the disk is idle or when running out of disk space

Solution 3 - Journaling (or Write-Ahead Logging)

Write "intent" down to disk before updating file system

- ➔ Called the "Write Ahead Logging" or "journal" originated from database community

When crash occurs, look through log to see what was going on

- Use contents of log to fix file system structures
- The process is called "recovery"

Case Study - Linux Ext3

Physical journaling - write real block contents of the update to log

1. Commit dirty blocks to journal as one transaction (TxBegin, inodes, bitmaps and data blocks)
2. Write commit record (TxEnd)
3. Copy dirty blocks to real file system (checkpointing)
4. Reclaim the journal space for the transaction

Logical journaling - write logical record of the operation to log

- "Add entry F to directory data block D"
- Complex to implement
- May be faster and save disk space

Ext3 - What if there is a crash

➔ **Recovery** - Go through log and "redo" operations that have been successfully committed to log

What if ...

- TxBegin but not TxEnd in log?
- TxBegin through TxEnd are in log, but inodes, bitmaps, and data have not yet been checkpointed?
- What if Tx is in log; inodes, bitmaps and data have been checkpointed; but Tx has not been freed from log?

Journaling Modes

Journaling has cost - one write = two disk writes (two seeks with hard disks)

➔ Several journaling modes balance consistency and performance

- **Data journalling** - journal all writes, including file data

- ◉ expensive to journal data

- **Metadata journaling** - journal only metadata

Used by most FS (IBM JFS, SGI XFS, NTFS)

- ◉ file may contain garbage data

- **Ordered mode** - write file data to real FS first, then journal metadata

Default mode for ext3

- ◉ old file may contain new data